VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

DATA ENGINEER

Supply Chain Management System

Advisor: Phan Trong Nhan
Authors: Nguyễn Đức Phú - 2570772
Bùi Bá Phúc - 2570799
Lưu Trọng Tơ - 2570802

HO CHI MINH CITY, APRIL 2026

Table Of Contents

1 Introduction **3**
1.1 Project Overview . . . 3

2 Requirements Gathering **4**
2.1 Stakeholders . . . 4
2.2 Business Processes . . . 4
2.3 Functional Requirements . . . 4
2.4 Non-Functional Requirements . . . 5
2.5 Business Rules . . . 5
2.6 Data Requirements . . . 6

3 Conceptual Design **6**
3.1 Core Entities . . . 6
3.2 Entity Relationships . . . 7
3.3 First ER Design . . . 7
3.4 Extended ER Design . . . 7
3.5 Conceptual ERD Notes . . . 8

4 Relational Schema as UML Class Diagram **8**

5 Database Design **10**
5.1 Table List . . . 10
5.1.1 Master Tables . . . 10
5.1.2 Transaction Tables . . . 10
5.2 Database Schema Diagram . . . 10
5.3 Transaction Table Specifications . . . 10
5.4 Normalization . . . 14
5.5 Integrity & Constraints . . . 15
5.5.1 Structural Constraints . . . 15
5.5.2 Cardinality and Participation Constraints . . . 15
5.5.3 Uniqueness and Domain Constraints . . . 15
5.5.4 Business Rules . . . 15

6 Data Warehouse Design **16**
6.1 Objectives . . . 16
6.2 Architecture . . . 16
6.3 Star Schema Design . . . 16
6.3.1 Dimension Table: Dim_Date . . . 16
6.3.2 Dimension Table: Dim_Sales_Order . . . 16
6.3.3 Fact Table: Fact_Delivery_Quality . . . 17
6.4 ETL Logic . . . 17
6.4.1 Step 1: Extraction . . . 17
6.4.2 Step 2: Transformation and Classification . . . 18
6.4.3 Step 3: Load . . . 18
6.5 KPI Aggregations . . . 18

7 Data Flow Design **19**
7.1 Context Diagram . . . 19
7.2 Level-1 Processes . . . 19
7.3 Overall Process Flowchart . . . 20
7.4 Evaluation of Streaming Services . . . 22
7.4.1 Technology Overview . . . 22
7.4.2 Comparative Evaluation . . . 22
7.4.3 Selection Discussion . . . 22
7.4.4 Benchmark Placeholder . . . 22
7.4.5 Conclusion . . . 23
7.5 Kafka CDC for Sale Orders . . . 23
7.5.1 Change Data Capture Overview . . . 23
7.5.2 CDC with the Outbox Pattern . . . 24
7.5.3 Benefits of CDC Integration . . . 24
7.5.4 Alternative Event Publication Techniques . . . 24
7.5.5 Comparative Summary . . . 25

8 Machine Learning Design **25**
8.1 Requirement Scope . . . 25
8.2 Design Objective . . . 25
8.3 High-Level Architecture . . . 25
8.4 Batch Training Pipeline . . . 26
8.4.1 Features . . . 26
8.4.2 Model Choice . . . 26
8.5 Model Registry and Artifact Management . . . 26
8.6 Streaming Inference Design . . . 27
8.7 Prediction Logging and Feedback Loop . . . 27
8.8 Supporting Data Structures . . . 27
8.9 Kafka Topics . . . 28

1 Introduction

1.1 Project Overview

- **Project Title:** Supply Chain Management System
- **Industry / Business Domain:** Supply Chain Operations, Warehouse Managements, Transportation and Logistics Trackings
- **Problem Description:** The organization requires an integrated internal platform to manage warehouse execution and transportation coordination in a more controlled, transparent, and efficient manner. Existing operations depend heavily on manual handling, paper-based records, and fragmented communication channels such as email, which limits operational visibility, increases processing time, and raises the risk of delays and execution errors across fulfillment and delivery activities.
- **Project Objectives:**
  - Streamline operational workflows by reducing manual intervention across warehouse and transportation processes.
  - Decrease reliance on paper-based procedures to lower administrative cost and support sustainability objectives.
  - Standardize and control end-to-end business processes in order to reduce errors and mitigate operational risk.
  - Improve fulfillment speed and delivery responsiveness to enhance customer experience.
  - Increase workforce productivity through better operational visibility and process coordination.
  - Establish a foundation for reporting, analytics, and ETA forecasting to support data-driven decision-making.
- **Project Scope:**
  - Manage vehicle requests submitted by order management teams through a centralized system.
  - Capture order information, customer details, delivery windows, and vehicle-capacity requirements.
  - Support dispatch review, approval, rejection, and assignment of internal or third-party vehicles and drivers.
  - Record operational events such as gate check-in, gate check-out, arrival timing, and delay reasons.
  - Enable drivers and warehouse personnel to update fulfillment and delivery progress in near real time.
  - Synchronize operational status, including Goods Issue updates, with enterprise systems such as ERP and eCommerce platforms.
  - Provide reporting dashboards, management insights, and predictive ETA capabilities.
  - Support tracking visibility for end users through real-time delivery monitoring services.
- **Out-of-Scope Items:**

- Replacement of the organization's ERP or eCommerce systems as core enterprise platforms.
- Unnecessary customizations that introduce vendor lock-in or limit future scalability.
- Business capabilities outside the scope of warehouse execution, dispatching, delivery tracking, and logistics reporting.

2 Requirements Gathering

2.1 Stakeholders

Role	Description
Drivers	Update delivery status, record gate check-in/check-out times, and capture delivery execution details.
Warehouse Workers	Pick, pack, and prepare eCommerce orders; update packing status and shipment status.
Regional Managers / Managers	View management reports, driver KPIs, fuel usage, and delivery performance analytics.
Fleet / Dispatch Manager	Review vehicle requests, assign internal or external vehicles and drivers, confirm or reject dispatch, and view warehouse arrival schedules.
End Users (Customers)	Monitor real-time delivery tracking status via the tracking service and receive ETA forecasts powered by an ML model consuming Kafka streaming events.

2.2 Business Processes

- Supplier onboarding
- Purchase requisition
- Purchase order processing
- Goods receipt
- Inventory management
- Order fulfillment
- Shipment & delivery
- Returns handling
- Reporting

2.3 Functional Requirements

- **Real-time tracking feature (Streaming Processing)**
  - R1: Allow drivers and warehouse workers to update tracking information.
  - R2: Allow end users to monitor tracking status.
  - R3: De-duplicate order requests.

- **Reporting (Batch Processing)**
  - R4: Create monthly and quarterly reports.
- **Operating features (Near real-time, low-latency CRUD)**
  - R5: Allow adding and removing drivers or transportation vendors.
- **Machine learning (Streaming Processing)**
  - R6: Allow ETA forecasting for end users based on historical data.

2.4 Non-Functional Requirements

- **ERP Integration:** The system shall integrate with the ERP platform to retrieve and synchronize customer, sales, and delivery information in a reliable and consistent manner.
- **Multi-Platform Accessibility:** The solution shall be accessible through both web and mobile applications to support operational users across different working environments.
- **Access Control and Security:** The system shall implement Role-Based Access Control (RBAC) to ensure that users can access only the functions and data permitted by their responsibilities.
- **Usability:** The user interface shall be intuitive, user-friendly, and designed to support efficient adoption by warehouse staff, drivers, dispatch teams, and managers.
- **Technology Flexibility:** The architecture shall minimize vendor lock-in risk to preserve long-term flexibility, maintainability, and scalability.
- **Auditability and Governance:** The system shall maintain user activity logs to support traceability, auditing, and governance compliance.

2.5 Business Rules

- The application shall support two customer operating models: **B2B** and **B2C**, each with its own order-handling and service expectations.
- A customer record shall be classified as either B2B or B2C, and this classification shall determine the required business attributes, fulfillment rules, and reporting views.
- B2B customers shall be associated with organization-level information such as company name, tax information, contractual terms, and business contact details.
- B2C customers shall be associated with individual-level information and shall follow simplified order and delivery handling rules designed for single-recipient fulfillment.
- B2B orders may include larger volume shipments, scheduled delivery windows, and dedicated dispatch approval depending on contractual or operational requirements.
- B2C orders shall prioritize fast fulfillment, granular tracking visibility, and customer-facing delivery updates, including ETA communication when available.
- Each sales order shall belong to exactly one customer and shall follow the process rules defined by that customer's classification.

- Each order item shall reference one valid product, and each product shall belong to one product category.
- Inventory shall be tracked at the combination of product and warehouse to maintain accurate stock visibility across fulfillment locations.
- The system shall not allow negative inventory balances unless an authorized override or exception workflow has been approved.
- Each delivery request shall reference a valid sales order, and each dispatch decision shall be linked to an assigned driver and vehicle or an approved external transportation arrangement.
- Returns shall reference the original fulfillment or delivery transaction to preserve traceability and support operational reconciliation.
- Duplicate order requests shall be identified and controlled before downstream fulfillment or dispatch activities are executed.
- Operational status updates entered by drivers or warehouse workers shall be time-stamped and recorded as auditable events.

2.6 Data Requirements

- Master Data
- Transactional Data
- Historical Data
- Reporting Data

3 Conceptual Design

3.1 Core Entities

Based on the extended EER diagram, the conceptual model is centered on the following entities:

- Customer
- Company
- Individual
- Sale Order
- Order Item
- Product
- Sale Order Status
- Delivery Order
- Shipment Request

- Warehouse
- B2B Request
- B2C Request
- Check-in Record
- Driver
- Vehicle
- Vendor

3.2 Entity Relationships

The extended EER diagram defines the following main conceptual relationships:

- Customer → Sale Order: a customer places one or more sales orders.
- Sale Order → Order Item: each sales order contains one or more order items.
- Order Item → Product: each order item refers to one product.
- Sale Order → Sale Order Status: a sales order has a status history across processing steps.
- Delivery Order → Sale Order: a delivery order fulfills one or more sales orders.
- Delivery Order → Shipment Request: each delivery order is requested through one shipment request.
- Delivery Order → Warehouse: each delivery order is dispatched from one warehouse.
- Shipment Request → B2B / B2C: each shipment request belongs to exactly one subtype.
- B2B Request → Driver: a B2B shipment is assigned to a driver.
- B2C Request → Vehicle: a B2C shipment is assigned to a vehicle.
- B2C Request → Check-in Record: a B2C request may generate a check-in record.
- Vehicle → Vendor: each vehicle belongs to one vendor.
- Customer → Company / Individual: customer is specialized into company and individual subtypes.

3.3 First ER Design

The first ER design of the system focuses on the sales domain. It models the main entities involved in customer order processing, including customer, sale order, order item, and product, using Chen notation.

3.4 Extended ER Design

The extended ER design presents the complete conceptual model in a horizontal layout for improved readability.

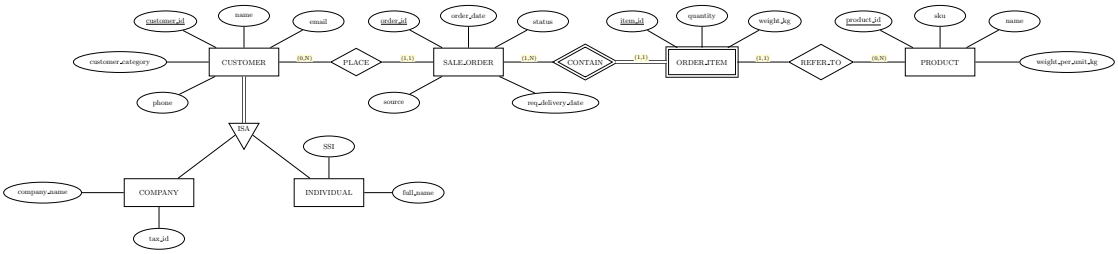

Figure 1: First ER design for the sales module

3.5 Conceptual ERD Notes

- **Customer generalization:** The entity `Customer` is specialized into `Individual` and `Company`. This specialization is **total** because every customer in the system must belong to one of these two business categories, and it is **disjoint** because a single customer record cannot be both an individual and a company at the same time. This rule follows the business requirement that the platform must manage both personal customers and corporate customers while keeping their identification attributes separate, such as full name and SSI for individuals, and company name and tax ID for companies.

- **Delivery Order to Sale Order:** The relationship from `Delivery Order` to `Sale Order` is designed so that one delivery order can fulfill multiple sales orders. This supports logistics efficiency by allowing the system to consolidate shipments that share similar destinations, schedules, or transport resources. At the same time, the current business rule assumes that one sales order cannot be split across multiple delivery orders. This is a deliberate simplification in the present scope, although we recognize that a more flexible future design could connect delivery orders to sale order lines instead of the whole sales order when partial fulfillment is required.

- **B2B and B2C specialization:** The entity `Shipment Request` is divided into `B2B` and `B2C` because these two fulfillment flows have different operational characteristics. B2B requests are associated with drivers and business delivery handling, while B2C requests are associated with vehicles, end-customer delivery, and optional check-in records. This specialization is **total** because every shipment request must follow either the B2B flow or the B2C flow, and it is **disjoint** because one request cannot belong to both flows simultaneously. This separation helps the conceptual model reflect the actual business processes and their distinct data requirements.

4 Relational Schema as UML Class Diagram

The relational schema derived from `sales.tex`, `packing.tex`, and `deliver.tex` is shown below as a UML-style class diagram. Each class represents a relation, the top compartment names the relation, the middle compartment lists attributes, and the bottom compartment highlights keys and foreign-key references.

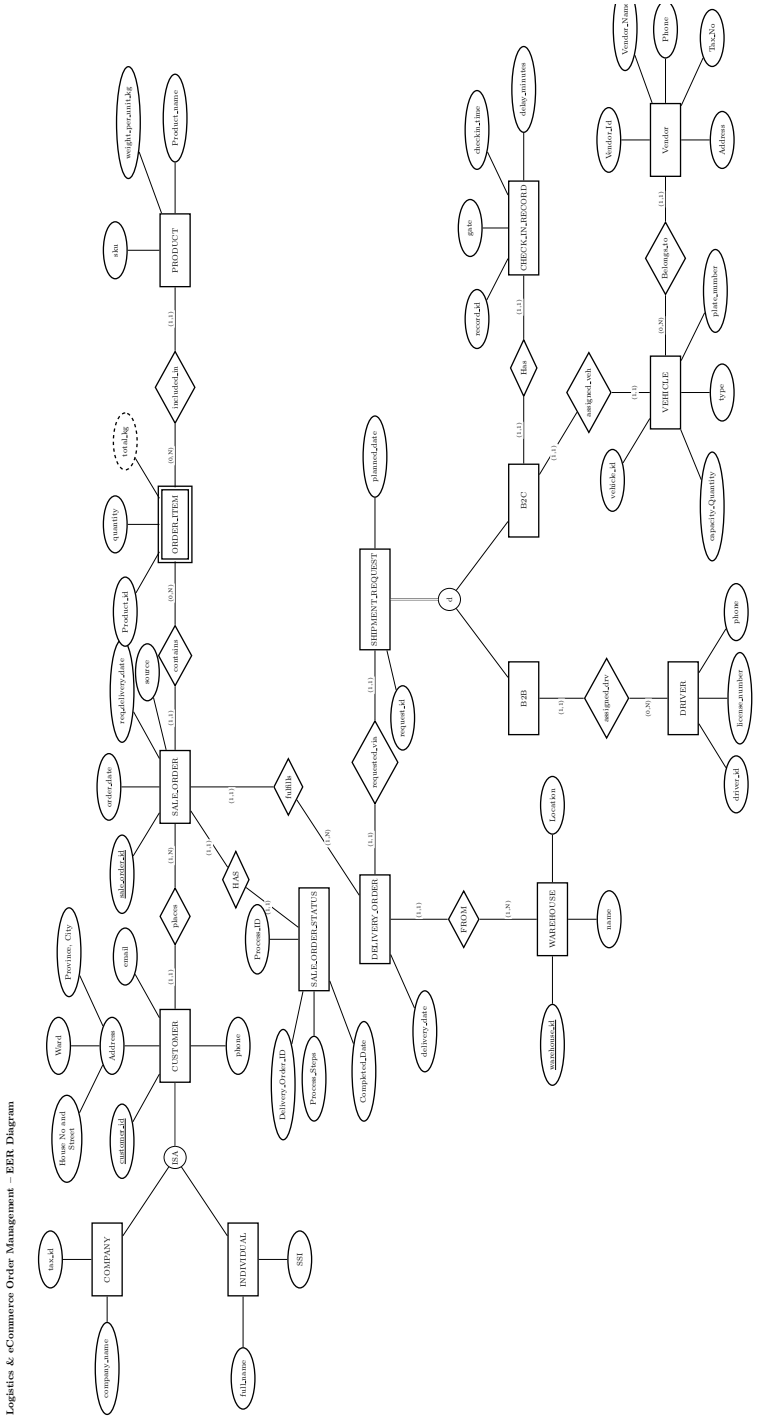

Figure 2: Extended ER design of the system

5 Database Design

5.1 Table List

5.1.1 Master Tables

- customers
- companies
- individuals
- products
- warehouses
- drivers
- vehicles
- vendors

5.1.2 Transaction Tables

- sale_orders
- order_items
- sale_order_status
- delivery_orders
- shipment_requests
- b2b_requests
- b2c_requests
- check_in_records

5.2 Database Schema Diagram

The database schema diagram for the application is presented below.

5.3 Transaction Table Specifications

`sale_orders`

- **Purpose:** Stores customer sales orders and links each order to the customer and the delivery batch that fulfills it.
- **Primary Key:** `sale_order_id`
- **Foreign Keys:** `customer_id` → `customers.customer_id`; `delivery_order_id` → `delivery_orders.deliver`
- **Columns:**

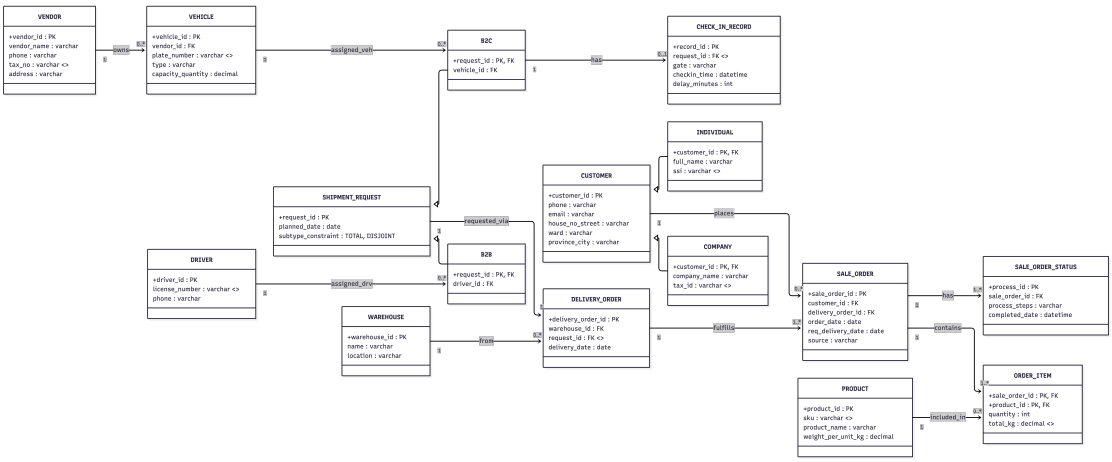

Figure 3: Database schema diagram

Column	Type	Description
sale_order_id	INT	Primary key for each sales order
customer_id	INT	References the customer placing the order
delivery_order_id	INT	References the assigned delivery order
order_date	DATE	Date when the sales order is created
req_delivery_date	DATE	Requested delivery date from the customer
status	VARCHAR	Current order state such as pending or delivered

- **Constraints:** `customer_id` and `delivery_order_id` are mandatory foreign keys; `order_date` $\leq$ `req_delivery_date`.
- **Indexes:** Primary key index on `sale_order_id`; non-unique indexes on `customer_id`, `delivery_order_id`, and `order_date`.

`order_items`

- **Purpose:** Stores the products and quantities included in each sales order.
- **Primary Key:** Composite key `(sale_order_id, product_id)`
- **Foreign Keys:** `sale_order_id` $\rightarrow$ `sale_orders.sale_order_id`; `product_id` $\rightarrow$ `products.product_id`
- **Columns:**

Column	Type	Description
sale_order_id	INT	Part of the composite primary key and reference to the order
product_id	INT	Part of the composite primary key and reference to the product
quantity	INT	Number of product units ordered
weight_per_unit_kg	DECIMAL	Weight per product unit in kilograms
total_kg	DECIMAL	Derived total weight for the line item

- **Constraints:** Composite primary key enforces one row per product per order; `quantity` > 0; `weight_per_unit_kg` > 0; `total_kg` is derived as `quantity` $\times$ `weight_per_unit_kg`.
- **Indexes:** Composite primary key index on `(sale_order_id, product_id)`; secondary index on `product_id`.

`sale_order_status`

- **Purpose:** Tracks the status history of each sales order over time.
- **Primary Key:** `status_id`
- **Foreign Keys:** `sale_order_id` $\rightarrow$ `sale_orders.sale_order_id`
- **Columns:**

Column	Type	Description
status_id	INT	Primary key for each status record
sale_order_id	INT	References the related sales order
status	VARCHAR	Status label such as confirmed, packed, or delivered
status_timestamp	DATETIME	Time when the status is recorded
remarks	VARCHAR	Optional note about the status update

- **Constraints:** `sale_order_id` is mandatory and each status row must belong to one sales order.
- **Indexes:** Primary key index on `status_id`; non-unique indexes on `sale_order_id` and `status_timestamp`.

`delivery_orders`

- **Purpose:** Groups shipment execution details for fulfilling one or more sales orders.
- **Primary Key:** `delivery_order_id`
- **Foreign Keys:** `warehouse_id` $\rightarrow$ `warehouses.warehouse_id`; `request_id` $\rightarrow$ `shipment_requests.request_id`
- **Columns:**

Column	Type	Description
delivery_order_id	INT	Primary key for each delivery order
warehouse_id	INT	References the warehouse handling dispatch
request_id	INT	References the associated shipment request
delivery_date	DATE	Actual or planned delivery date
status	VARCHAR	Delivery progress status

- **Constraints:** `warehouse_id` and `request_id` are mandatory foreign keys; `request_id` must be unique to preserve the one-to-one relationship with `shipment_requests`.
- **Indexes:** Primary key index on `delivery_order_id`; unique index on `request_id`; non-unique index on `warehouse_id`.

`shipment_requests`

- **Purpose:** Stores the common request information shared by both B2B and B2C shipment processes.
- **Primary Key:** `request_id`
- **Foreign Keys:** None
- **Columns:**

Column	Type	Description
request_id	INT	Primary key for each shipment request
request_type	VARCHAR	Distinguishes B2B from B2C requests
request_date	DATE	Date when the request is submitted
origin	VARCHAR	Shipment origin or dispatch point
destination	VARCHAR	Shipment destination

- **Constraints:** Every request must appear in exactly one subtype table, either `b2b_requests` or `b2c_requests`.
- **Indexes:** Primary key index on `request_id`; non-unique indexes on `request_type` and `request_date`.

`b2b_requests`

- **Purpose:** Stores B2B-specific shipment request details and assigned driver information.
- **Primary Key:** `request_id`
- **Foreign Keys:** `request_id` → `shipment_requests.request_id`; `driver_id` → `drivers.driver_id`
- **Columns:**

Column	Type	Description
request_id	INT	Primary key and inherited reference to shipment request
driver_id	INT	References the driver assigned to the shipment
loading_dock	VARCHAR	Loading location used for the shipment
dispatch_time	DATETIME	Time when the B2B shipment departs

- **Constraints:** `request_id` must already exist in `shipment_requests`; subtype membership is total and disjoint.
- **Indexes:** Primary key and unique index on `request_id`; non-unique index on `driver_id`.

`b2c_requests`

- **Purpose:** Stores B2C-specific shipment request details and assigned vehicle information.
- **Primary Key:** `request_id`
- **Foreign Keys:** `request_id` → `shipment_requests.request_id`; `vehicle_id` → `vehicles.vehicle_id`

- **Columns:**

Column	Type	Description
request_id	INT	Primary key and inherited reference to shipment request
vehicle_id	INT	References the vehicle assigned to the shipment
recipient_name	VARCHAR	Name of the delivery recipient
contact_number	VARCHAR	Recipient contact information
scheduled_time	DATETIME	Planned delivery time window

- **Constraints:** `request_id` must already exist in `shipment_requests`; subtype membership is total and disjoint.
- **Indexes:** Primary key and unique index on `request_id`; non-unique index on `vehicle_id`.

`check_in_records`

- **Purpose:** Records customer-side check-in activity for B2C delivery fulfillment.
- **Primary Key:** `check_in_id`
- **Foreign Keys:** `request_id` → `b2c_requests.request_id`
- **Columns:**

Column	Type	Description
check_in_id	INT	Primary key for each check-in record
request_id	INT	References the related B2C request
check_in_time	DATETIME	Time when the check-in is recorded
delay_minutes	INT	Delay duration measured in minutes
notes	VARCHAR	Optional remarks captured during check-in

- **Constraints:** `request_id` must be unique; `delay_minutes` ≥ 0; this table applies only to the B2C process.
- **Indexes:** Primary key index on `check_in_id`; unique index on `request_id`; non-unique index on `check_in_time`.

5.4 Normalization

- 1NF: passed
- 2NF: passed
- 3NF: passed
- Denormalization (if needed): derived attribute `total_kg` in the `order_items` table may be stored to optimize performance.

5.5 Integrity & Constraints

The detailed primary key and referential relationships are already presented in the database schema diagram in Figure 3. To avoid repetition, this subsection summarizes only the main integrity rules and business-level constraints that guide implementation.

5.5.1 Structural Constraints

- The relation `ORDER_ITEM` uses a composite primary key: `(sale_order_id, product_id)`.
- Subtype tables inherit the identifier of their parent entity, including `COMPANY` and `INDIVIDUAL` from `CUSTOMER`, and `B2B` and `B2C` from `SHIPMENT_REQUEST`.
- The `CUSTOMER` hierarchy is **total** and **disjoint**: every customer must be either a `COMPANY` or an `INDIVIDUAL`, but not both.
- The `SHIPMENT_REQUEST` hierarchy is **total** and **disjoint**: every shipment request must be either `B2B` or `B2C`, but not both.

5.5.2 Cardinality and Participation Constraints

- A customer may create multiple sales orders, but each sales order belongs to exactly one customer.
- A delivery order may serve multiple sales orders, while each sales order belongs to exactly one delivery order.
- A sales order may contain multiple order items and multiple status records.
- A shipment request has a one-to-one relationship with a delivery order.
- A B2C request may have zero or one associated check-in record.

5.5.3 Uniqueness and Domain Constraints

- Unique business identifiers include `PRODUCT.sku`, `COMPANY.tax_id`, `INDIVIDUAL.ssi`, `DRIVER.license_number`, `VEHICLE.plate_number`, and `VENDOR.tax_no`.
- `DELIVERY_ORDER.request_id` and `CHECK_IN_RECORD.request_id` must be unique.
- `quantity` > 0, `weight_per_unit_kg` > 0, `capacity_quantity` > 0, and `delay_minutes` ≥ 0.
- `order_date` $\leq$ `req_delivery_date`, and `delivery_date` $\geq$ `order_date`.
- The derived attribute `ORDER_ITEM.total_kg` is computed as `quantity` $\times$ `weight_per_unit_kg`.

5.5.4 Business Rules

- Each `SHIPMENT_REQUEST` must appear in exactly one subtype: `B2B` or `B2C`.
- `CHECK_IN_RECORD` applies only to the `B2C` process.
- Entity integrity, key integrity, and referential integrity are enforced according to the schema diagram and the relational definitions above.

The relational design therefore enforces entity integrity, referential integrity, uniqueness rules, domain validation, specialization constraints, and operational business rules in a consistent manner.

6 Data Warehouse Design

6.1 Objectives

The data warehouse for this system is designed to support delivery quality analytics and KPI tracking based on transactional data sourced from the `sale_orders` and `sale_order_status` tables. The specific objectives are:

- Track and classify delivery performance per order on a monthly basis.
- Calculate on-time delivery rates and late delivery distributions.
- Support management reporting through pre-aggregated, query-optimized fact data.

6.2 Architecture

Source Systems → Staging → ETL → Data Warehouse → BI Tools

The source system is the operational PostgreSQL database. The staging layer extracts raw transactional records. The ETL layer applies transformation and classification logic before loading into the star schema. BI tools query the warehouse layer for reporting and dashboards.

6.3 Star Schema Design

The grain of this star schema is one row per sales order. The schema consists of one fact table and two dimension tables, derived entirely from `sale_orders` and `sale_order_status`.

6.3.1 Dimension Table: Dim_Date

This table supports time-based aggregations, specifically monthly grouping for KPI calculations such as total orders per month.

Column	Type	Description
Date_Key	INT	Surrogate primary key for each date record
Full_Date	DATE	The complete calendar date value
Month	INT	Numeric month (1–12)
Year	INT	Four-digit calendar year
Year_Month	VARCHAR	Formatted label for monthly grouping, e.g. `'2025-10'`

Table 1: Dim_Date column definitions

6.3.2 Dimension Table: Dim_Sales_Order

Contains descriptive attributes of each order extracted from the `sale_orders` table.

Column	Type	Description
Order_Key	INT	Surrogate primary key for the dimension row
Sale_Order_ID	INT	Business key sourced from `sale_orders.sale_order_id`
Current_Status	VARCHAR	Latest status of the order at the time of ETL load

Table 2: Dim_Sales_Order column definitions

6.3.3 Fact Table: Fact_Delivery_Quality

The grain is one row per sales order. Pre-calculated metrics and binary classification flags are stored here to make reporting queries highly performant.

Column	Type	Description
Order_Key	INT	Foreign key to Dim_Sales_Order
Order_Date_Key	INT	Foreign key to Dim_Date for the order creation date
Planned_Delivery_Date_Key	INT	Foreign key to Dim_Date for `req_delivery_date`
Actual_Delivery_Date_Key	INT	Foreign key to Dim_Date for the delivered timestamp; nullable for undelivered orders
Delay_Days	INT	Actual delivery date minus planned delivery date in days; null when not yet delivered
Is_Not_Yet_Delivered	INT	1 if the order has no delivered status in the reporting month, 0 otherwise
Is_On_Time	INT	1 if Delay_Days ≤ 0, 0 otherwise
Is_Late_Under_1_Week	INT	1 if Delay_Days is between 1 and 7 inclusive, 0 otherwise
Is_Late_Under_2_Weeks	INT	1 if Delay_Days is between 8 and 14 inclusive, 0 otherwise
Is_Late_Over_2_Weeks	INT	1 if Delay_Days is greater than 14, 0 otherwise

Table 3: Fact_Delivery_Quality column definitions

6.4 ETL Logic

6.4.1 Step 1: Extraction

The base dates are extracted from the two source tables. The actual delivery date is derived from `sale_order_status` by selecting the timestamp of the `'delivered'` status entry.

```
SELECT
    so.sale_order_id,
    so.order_date,
    so.req_delivery_date            AS planned_delivery_date,
    MAX(
        CASE
            WHEN sos.status = 'delivered'
```

```
            THEN sos.status_timestamp
        END
    )                                AS actual_delivery_date
FROM sale_orders so
LEFT JOIN sale_order_status sos
    ON so.sale_order_id = sos.sale_order_id
GROUP BY
    so.sale_order_id,
    so.order_date,
    so.req_delivery_date;
```

6.4.2 Step 2: Transformation and Classification

Using the extracted dates, `Delay_Days` is computed and each binary classification flag is populated for every row loaded into the fact table.

1. **Delay calculation:**

   ```
   Delay_Days = actual_delivery_date::date
              - planned_delivery_date::date
   ```

2. **Is_Not_Yet_Delivered:** `actual_delivery_date IS NULL` and the order date falls within the current reporting month.
3. **Is_On_Time:** `actual_delivery_date IS NOT NULL` and `Delay_Days` ≤ 0.
4. **Is_Late_Under_1_Week:** `actual_delivery_date IS NOT NULL` and `Delay_Days` > 0 and `Delay_Days` ≤ 7.
5. **Is_Late_Under_2_Weeks:** `actual_delivery_date IS NOT NULL` and `Delay_Days` > 7 and `Delay_Days` ≤ 14.
6. **Is_Late_Over_2_Weeks:** `actual_delivery_date IS NOT NULL` and `Delay_Days` > 14.

The classification flags are mutually exclusive and collectively exhaustive across all delivered orders. Each order receives exactly one flag set to 1, or `Is_Not_Yet_Delivered` is set to 1 if no delivery has occurred within the reporting period.

6.4.3 Step 3: Load

The transformed and classified rows are loaded into `Fact_Delivery_Quality`. Date keys for `Order_Date_Key`, `Planned_Delivery_Date_Key`, and `Actual_Delivery_Date_Key` are resolved by joining against `Dim_Date` on `Full_Date`. `Order_Key` is resolved by joining against `Dim_Sales_Order` on `Sale_Order_ID`.

6.5 KPI Aggregations

Once data is loaded into `Fact_Delivery_Quality`, all KPI calculations reduce to simple column summations grouped by the date dimension. Complex timestamp matching and interval arithmetic are handled entirely in the ETL phase.

KPI	Aggregation Logic
Total Orders per Month	SUM(Is_Not_Yet_Delivered) + SUM(Is_On_Time) + SUM(Is_Late_Under_1_Week) + SUM(Is_Late_Under_2_Weeks) + SUM(Is_Late_Over_2_Weeks)
Delivery Successful	SUM(Is_On_Time) + SUM(Is_Late_Under_1_Week) + SUM(Is_Late_Under_2_Weeks) + SUM(Is_Late_Over_2_Weeks)
On-Time Delivery Rate	SUM(Is_On_Time) / Total Orders per Month
Not Yet Delivered	SUM(Is_Not_Yet_Delivered)
Late Under 1 Week Rate	SUM(Is_Late_Under_1_Week) / Total Orders per Month
Late Under 2 Weeks Rate	SUM(Is_Late_Under_2_Weeks) / Total Orders per Month
Late Over 2 Weeks Rate	SUM(Is_Late_Over_2_Weeks) / Total Orders per Month

Table 4: KPI aggregation formulas over Fact_Delivery_Quality

All aggregations are filtered by joining `Fact_Delivery_Quality` to `Dim_Date` on `Order_Date_Key` and applying a `WHERE` clause on `Dim_Date.Year_Month` to scope results to the desired reporting period.

7 Data Flow Design

7.1 Context Diagram

External Entities:

- Supplier
- Customer
- Warehouse
- Logistics Provider
- BI Users

7.2 Level-1 Processes

Process 1: Supplier Management

- **Input:** Supplier data
- **Output:** Approved supplier
- **Data Store:** Supplier Master

Process 2: Procurement

- **Input:** Requisition
- **Output:** Purchase Order
- **Data Store:** PO Tables

Process 3: Inventory Update

- **Input:** Goods Receipt
- **Output:** Updated Inventory
- **Data Store:** Inventory Tables

Process 4: Order Fulfillment

- **Input:** Sales Order
- **Output:** Shipment
- **Data Store:** Sales + Shipment Tables

Process 5: Returns

- **Input:** Return Request
- **Output:** Inventory Adjustment
- **Data Store:** Return Tables

Process 6: Reporting

- **Input:** Operational Data
- **Output:** Reports / Dashboards
- **Data Store:** Data Warehouse

7.3 Overall Process Flowchart

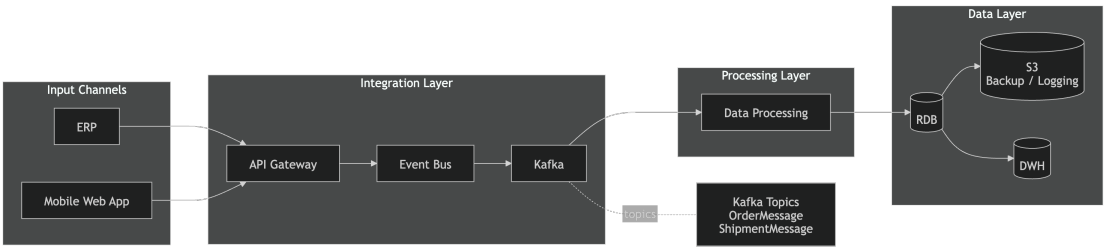

Figure 4: Component view of overall architecture

Figure 4 illustrates the main components involved in requirement **R6**. It highlights the separation between the offline training path, which produces and registers the model, and the online inference path, which consumes streaming shipment events and emits ETA predictions to downstream services.

Figure 5 summarizes the main operational flow of the system. It shows how the process begins with supplier and procurement activities, continues through inventory updates and sales order fulfillment, and finally feeds reporting and monitoring functions. This flowchart complements the Level-1 process descriptions by presenting the sequence of major activities in a single visual view.

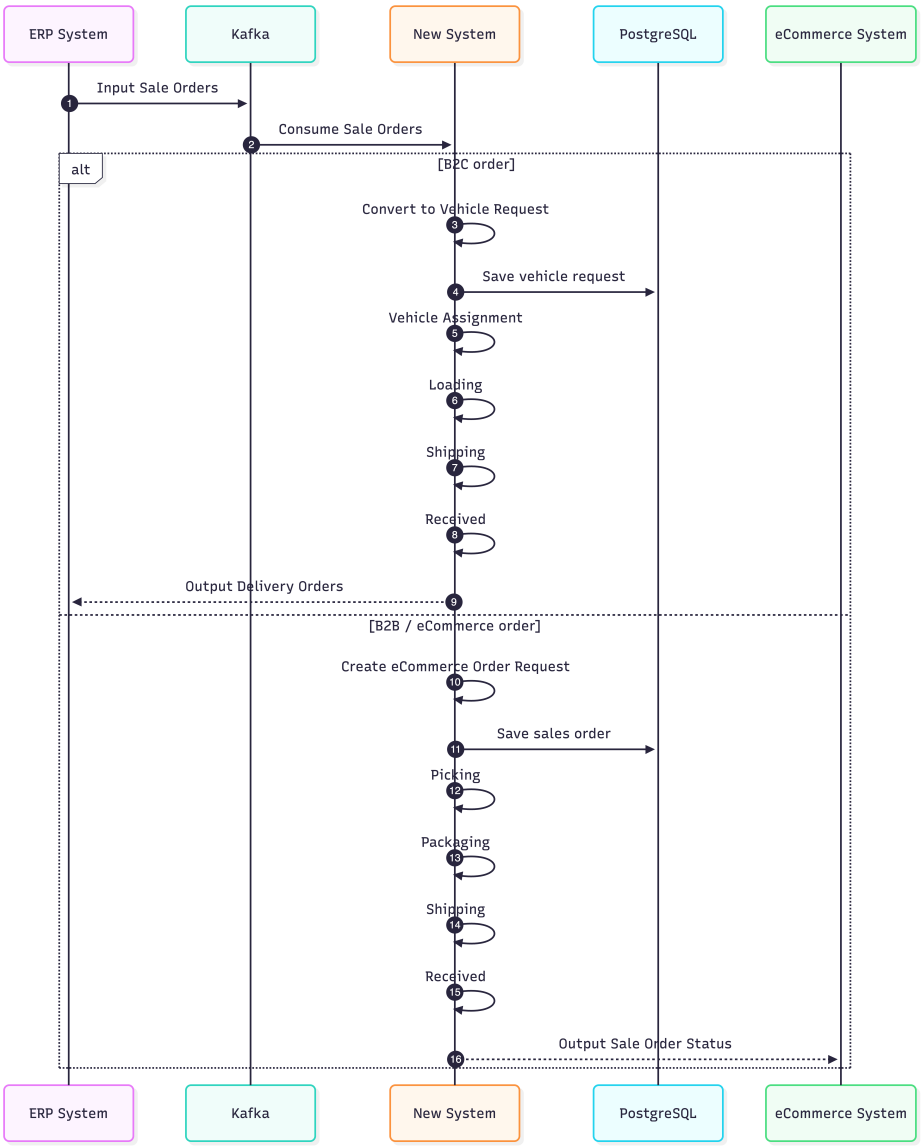

Figure 5: Overall operational flowchart of the supply chain system

7.4 Evaluation of Streaming Services

To support event-driven data movement in the supply chain system, it is useful to compare several messaging and streaming technologies before selecting a final platform. This section briefly evaluates Apache Kafka, RabbitMQ, and MQTT with respect to their suitability for operational events such as order updates, delivery notifications, and downstream analytics feeds.

7.4.1 Technology Overview

7.4.1.1 Apache Kafka. Kafka is a distributed event streaming platform designed for high-throughput and durable log-based messaging. It is well suited for CDC pipelines, analytics integration, and scalable event replay.

7.4.1.2 RabbitMQ. RabbitMQ is a message broker that supports flexible routing, acknowledgements, and queue-based communication. It is often appropriate for task dispatching, transactional workflows, and service-to-service messaging patterns.

7.4.1.3 MQTT. MQTT is a lightweight publish-subscribe protocol optimized for low-bandwidth and resource-constrained environments. It is commonly used for IoT devices, sensors, and edge communication scenarios.

7.4.2 Comparative Evaluation

Technology	Primary Strength	Typical Use Case	Limitation in This Project
Kafka	High throughput and durable event logs	CDC, analytics streaming, asynchronous event pipelines	More operational complexity than a traditional broker
RabbitMQ	Flexible routing and reliable queues	Workflow messaging, command delivery, service orchestration	Less suitable for large-scale log replay and stream retention
MQTT	Lightweight transport and low overhead	Device telemetry, mobile or edge notifications	Limited for large transactional stream processing workloads

7.4.3 Selection Discussion

For this project, Kafka is the most appropriate choice for database change streaming because it aligns well with the CDC plus outbox architecture described later in this section. RabbitMQ remains a strong candidate for short-lived operational messaging, while MQTT is more appropriate if the system is later extended to include sensors, vehicle devices, or warehouse IoT endpoints.

7.4.4 Benchmark Placeholder

Table 5 provides a dummy benchmark template for future experimental evaluation. The values are placeholders only and should be replaced with measured results collected under the same workload conditions.

Technology	Avg. Latency (ms)	Throughput (msg/s)
Kafka	12.0	50000
RabbitMQ	8.5	18000
MQTT	5.0	12000

Table 5: Placeholder benchmark for latency and throughput comparison of streaming services

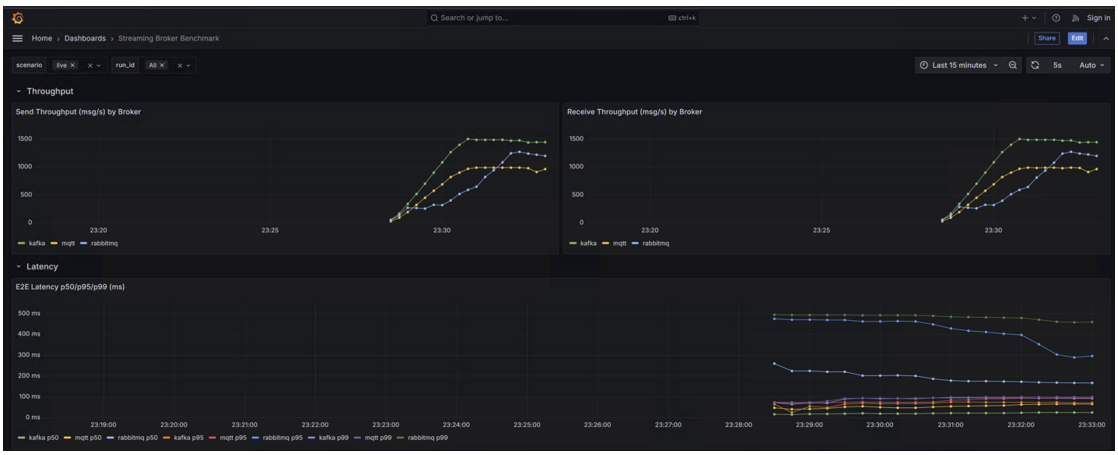

Figure 6: Benchmark reference snapshot for latency and throughput comparison

Figure 6 is included as a reference snapshot to support the placeholder benchmark table above. It serves as illustrative proof of the benchmarking concept and can be replaced later with final measured experimental output.

7.4.5 Conclusion

The preliminary comparison suggests that Kafka is the strongest candidate for the main event streaming backbone of the system, especially for `SALE_ORDER` CDC events. RabbitMQ and MQTT are still relevant alternatives for different messaging patterns, and the benchmark template provides a basis for later empirical validation.

7.5 Kafka CDC for Sale Orders

In the integration between PostgreSQL and Apache Kafka, maintaining consistency between transactional updates and published events is a central design concern. For the `SALE_ORDER` process, this report adopts an outbox-based event publication strategy strengthened by Change Data Capture (CDC). This approach avoids the dual-write problem and supports reliable event-driven processing across downstream services.

7.5.1 Change Data Capture Overview

Change Data Capture is a mechanism for detecting database changes, including inserts, updates, and deletes, and forwarding them to external systems with minimal delay. In PostgreSQL, CDC is typically implemented by reading the Write-Ahead Log (WAL) through logical replication. A

CDC connector such as Debezium can then transform committed database changes into Kafka events.

7.5.2 CDC with the Outbox Pattern

For `SALE_ORDER`, the application writes both the business record and a corresponding `ORDER_OUTBOX` record in the same database transaction. After the transaction commits, the CDC connector reads the outbox change from WAL, converts it into a structured event, and publishes it to the appropriate Kafka topic. Because the event is derived from committed database state, the design preserves transactional correctness while removing the need for manual polling.

Step	Description
Application write	Insert or update `SALE_ORDER` and insert a matching `ORDER_OUTBOX` row in one transaction
Commit	PostgreSQL commits the transaction and persists the change to WAL
CDC capture	CDC reads the committed outbox record from the WAL stream
Kafka publish	The connector publishes the resulting event to Kafka for downstream consumers

7.5.3 Benefits of CDC Integration

Benefit	Explanation
Low latency	Events are propagated to Kafka in near real time after transaction commit
Efficiency	The system avoids repeated polling queries on the outbox table
Scalability	Log-based capture supports larger workloads more effectively than periodic scanning
Consistency	Kafka events reflect committed database state and therefore preserve correctness

7.5.4 Alternative Event Publication Techniques

Several alternative techniques may be used to move database changes to Kafka, but they offer different trade-offs in terms of reliability, latency, and operational complexity.

7.5.4.1 Direct dual write. The application writes to PostgreSQL and publishes to Kafka separately. This design is simple, but it does not guarantee atomicity. If one action succeeds and the other fails, the system may produce inconsistent state or missing events.

7.5.4.2 Polling-based outbox. The application writes to an outbox table, and a background worker periodically queries for unpublished rows. This solution is easy to implement and does not require extra streaming infrastructure, but it introduces polling delay and additional query overhead under heavy load.

7.5.4.3 CDC-based outbox. The application writes to the outbox table, while CDC streams committed changes directly to Kafka. This design combines strong consistency with low latency and is the preferred approach for the `SALE_ORDER` workflow.

7.5.4.4 Trigger-based publishing. Database triggers react to table changes and publish events or push them into an intermediate queue. Although the response can be immediate, this technique tightly couples integration logic to the database layer and may negatively affect transaction performance and maintainability.

7.5.4.5 CDC without outbox. A CDC tool may read changes directly from business tables such as `SALE_ORDER`. This removes the need for an outbox table, but it weakens control over event semantics because physical table updates do not always map cleanly to business events.

7.5.5 Comparative Summary

Approach	Consistency	Latency	Complexity	Assessment
Direct dual write	Low	Low	Low	Not recommended
Polling outbox	High	Medium	Low	Acceptable
CDC plus outbox	High	Low	Medium	Preferred
Trigger-based publishing	Medium	Low	High	Not preferred
CDC without outbox	Medium	Low	Medium	Limited use

8 Machine Learning Design

8.1 Requirement Scope

Requirement **R6** states that the system shall support ETA forecasting for end users based on historical shipment data. To satisfy this requirement, the proposed design combines offline model training with online event-driven inference so that predicted delivery times can be generated for newly created shipments and exposed to downstream services.

8.2 Design Objective

The machine learning component has two goals:

- train an ETA prediction model from historical shipment records,
- serve real-time ETA estimates when shipment events are created in the operational system.

This design separates training from inference. Training is performed in batch mode on accumulated historical data, while inference is executed continuously on streaming events. Such separation improves maintainability, scalability, and operational clarity.

8.3 High-Level Architecture

The proposed pipeline is organized as follows:

`PostgreSQL` → `Python training pipeline` → `MLflow/MinIO` → `Faust worker (Kafka)` → `eta.predicted`

In this architecture, PostgreSQL stores the historical shipment data used for model training. A Python-based training process extracts features, trains a model, and logs the resulting artifact to MLflow. The production model is then loaded by a Kafka-based streaming worker implemented with Faust, which consumes shipment creation events and publishes ETA prediction events.

8.4 Batch Training Pipeline

The batch training pipeline is executed on a schedule or on demand. Its main steps are:

1. query completed shipment records with known delivery outcomes from PostgreSQL,
2. construct training features and labels,
3. train the ETA prediction model,
4. log parameters, metrics, and artifacts to MLflow,
5. promote the best-performing model to the production stage.

Historical data is used because ETA prediction depends on observed delivery durations under different operational conditions. The target label is the actual delivery duration, computed as the time difference between shipment creation and delivery completion.

8.4.1 Features

Representative features for ETA forecasting include:

Feature	Source	Type
`carrier`	shipment carrier information	Categorical
`channel`	order sales channel	Categorical
`item_count`	number of items in the order	Numeric
`total_weight_kg`	package or shipment weight	Numeric
`day_of_week`	derived from shipment creation time	Numeric
`hour_of_day`	derived from shipment creation time	Numeric
`actual_eta_hours`	delivered_at − created_at	Label

8.4.2 Model Choice

Tree-based models such as LightGBM or scikit-learn gradient boosting are appropriate for this problem because shipment ETA prediction is based on structured tabular data rather than images or free text. These models train quickly, require no GPU infrastructure, generate compact model artifacts, and provide interpretable feature importance values for evaluation.

8.5 Model Registry and Artifact Management

MLflow is used for experiment tracking and model versioning. It records training parameters, evaluation metrics, and model artifacts for each run. MinIO acts as the artifact store for trained models, while metadata remains accessible through MLflow.

This configuration supports a controlled promotion process such as `Staging` to `Production`. As a result, inference services always load a versioned and auditable model rather than an unmanaged local file.

8.6 Streaming Inference Design

Once a production model is available, a Faust worker consumes Kafka events from the shipment creation stream. For each new shipment event, the worker extracts features from the event payload, applies the loaded model, and publishes the predicted ETA to a dedicated Kafka topic.

Step	Description
Consume event	Read a `shipment.created` message from Kafka
Extract features	Build the feature vector required by the ETA model
Run inference	Execute `model.predict()` in the worker process
Publish result	Write the ETA prediction to `eta.predicted`
Persist prediction	Store prediction metadata for later evaluation

This design allows real-time ETA estimation without coupling online inference to the batch training job. The streaming worker only depends on the latest promoted model artifact.

8.7 Prediction Logging and Feedback Loop

To support continuous improvement, predictions must be stored and later compared with actual outcomes. When a shipment reaches the delivered state, the system can retrieve the original prediction, compute the prediction error, and store the result for future retraining and monitoring.

This feedback loop can be summarized as follows:

`Train` → `Deploy` → `Predict` → `Observe actual outcome` → `Measure error` → `Retrain`

Without this step, the system would not be able to evaluate model drift or verify that ETA predictions remain useful over time.

8.8 Supporting Data Structures

The machine learning module requires at least two operational tables for prediction storage and accuracy tracking.

`predictions`

Column	Type
`id`	UUID PK
`shipment_id`	UUID FK
`predicted_eta_hours`	FLOAT
`model_version`	VARCHAR
`input_features`	JSONB
`predicted_at`	TIMESTAMP

`prediction_actuals`

Column	Type
`id`	UUID PK
`shipment_id`	UUID FK
`prediction_id`	UUID FK
`actual_eta_hours`	FLOAT
`absolute_error`	FLOAT
`recorded_at`	TIMESTAMP

These tables provide the minimum structure needed to support inference logging, post-delivery evaluation, and retraining data collection.

8.9 Kafka Topics

The ETA forecasting workflow uses the following Kafka topics:

Topic	Producer	Consumer
`shipment.created`	Shipment service	Faust inference worker
`eta.predicted`	Faust inference worker	Notification and dashboard services

This design reuses the existing shipment creation stream and only introduces a dedicated prediction output topic.